

OS-QA

Chapter 1: Introduction

1.1 What are the **three main purposes** of an operating system?

中文：操作系统的三个主要目的是什么？

Answer (English):

The three main purposes are:

- To **provide an environment for a computer user** to execute programs on computer hardware in a convenient and efficient manner.
- To **allocate the separate resources** of the computer as needed to perform the required tasks. The allocation process should be as fair and efficient as possible.
- As a **control program**, it serves two major functions: (1) **supervision of the execution of user programs** to prevent errors and improper use of the computer, and (2) **management of the operation and control of I/O devices**.

答案（中文）：

三个主要目的是：

- 为计算机用户提供一个方便高效地在计算机硬件上执行程序的环境。
- 根据需要分配计算机的各个资源以执行所需任务，分配过程应尽可能公平和高效。
- 作为控制程序，它有两个主要功能：(1) 监督用户程序的执行，防止错误和不当使用计算机；(2) 管理I/O设备的操作和控制。

1.2 We have stressed the need for an operating system to make efficient use of the computing hardware. When is it **appropriate for the operating system to forsake this principle and to "waste" resources**? Why is such a system not really wasteful?

中文：我们强调了操作系统需要有效利用计算硬件。在什么情况下操作系统可以放弃这一原则而“浪费”资源？为什么这样的系统实际上并不是真的浪费？

Answer (English):

Single-user systems should **maximize use of the system for the user**. A **GUI** might "waste" CPU cycles, but it optimizes the user's interaction with the system.

答案（中文）：

单用户系统应为用户最大化系统使用。图形用户界面可能“浪费”CPU周期，但它优化了用户与系统的交互，因此并非真正浪费。

1.3 What is the main difficulty that a programmer must overcome in writing an operating system for a real-time environment?

中文：为实时环境编写操作系统时，程序员必须克服的主要困难是什么？

Answer (English):

The main difficulty is keeping the operating system within the fixed time constraints of a real-time system. If the system does not complete a task in a certain time frame, it may cause a breakdown of the entire system.

答案（中文）：

主要困难是将操作系统保持在实时系统的固定时间约束内。如果系统未能在特定时间范围内完成任务，可能导致整个系统崩溃。

1.4 Keeping in mind the various definitions of operating system, consider whether the operating system should include applications such as web browsers and mail programs. Argue both that it should and that it should not, and support your answers.

中文：考虑到操作系统的各种定义，请考虑操作系统是否应包含Web浏览器和邮件程序等应用程序。分别论证应该和不应该的理由，并支持你的答案。

Answer (English):

An argument in favor of including popular applications in the operating system is that if the application is embedded within the operating system, it is likely to be better able to take advantage of features in the kernel and therefore have performance advantages over an application that runs outside of the kernel.

Arguments against embedding applications within the operating system typically dominate, however: (1) the applications are applications—not part of an operating system, (2) any performance benefits of running within the kernel are offset by security vulnerabilities, and (3) inclusion of applications leads to a bloated operating system.

答案（中文）：

支持将流行应用包含在操作系统中的理由是，如果应用嵌入操作系统，它可能能更好地利用内核特性，从而比运行在内核外的应用有性能优势。

然而，反对嵌入的理由通常更占主导：(1) 这些应用是应用程序，不是操作系统的一部分；(2) 在内核中运行的任何性能优势都被安全漏洞抵消；(3) 包含应用会导致操作系统臃肿。

1.5 How does the distinction between kernel mode and user mode function as a rudimentary form of protection (security)?

中文：内核模式和用户模式之间的区别如何作为一种基本的保护（安全）形式发挥作用？

Answer (English):

The distinction between kernel mode and user mode provides a rudimentary form of protection in the following manner.

Certain instructions can be executed only when the CPU is in kernel mode. Similarly, hardware devices can be accessed only when the program is in kernel mode, and interrupts can be enabled or disabled only when the CPU is in kernel mode. Consequently, the CPU has very limited capability when executing in user mode, thereby enforcing protection of critical resources.

答案（中文）：

内核模式和用户模式的区别以以下方式提供基本保护。某些指令只能在CPU处于内核模式时执行。类似地，硬件设备只能在程序处于内核模式时访问，中断只能在内核模式下启用或禁用。因此，CPU在用户模式下执行时能力非常有限，从而实现对关键资源的保护。

1.6 Which of the following instructions should be privileged?

- a. Set value of timer.
- b. Read the clock.
- c. Clear memory.
- d. Issue a trap instruction.
- e. Turn off interrupts.
- f. Modify entries in device-status table.
- g. Switch from user to kernel mode.
- h. Access I/O device.

中文：以下哪些指令应该是特权指令？

- a. 设置定时器值。
- b. 读取时钟。
- c. 清空内存。
- d. 发出陷阱指令。
- e. 关闭中断。
- f. 修改设备状态表中的条目。
- g. 从用户模式切换到内核模式。
- h. 访问I/O设备。

Answer (English):

The following operations need to be privileged: set value of timer, clear memory, turn off interrupts, modify entries in device-status table, access I/O device. The rest can be performed in user mode.

答案（中文）：

以下操作需要特权：设置定时器值、清空内存、关闭中断、修改设备状态表条目、访问I/O设备。其余可以在用户模式下执行。

1.7 Some early computers protected the operating system by placing it in a memory partition that could not be modified by either the user job or the operating system itself. Describe two difficulties that you think could arise with such a scheme.

中文：一些早期计算机通过将操作系统放在一个用户作业或操作系统本身都无法修改的内存分区中来保护操作系统。描述你认为这种方案可能引起的两个困难。

Answer (English):

The data required by the operating system (passwords, access controls, accounting information, and so on) would have to be stored in or passed through unprotected memory and thus be accessible to unauthorized users. Further more, if OS itself cannot modify the code, it cannot install the patch and renew.

答案（中文）：

操作系统所需的数据（密码、访问控制、记账信息等）将不得不存储在未受保护的内存中或通过未受保护的内存传递，从而可能被未授权用户访问。另外，如果操作系统本身不能修改其代码，则无法安装补丁或更新。

1.8 Some CPUs provide for more than two modes of operation. What are two possible uses of these multiple modes?

中文：一些CPU提供多于两种操作模式。这些多模式有哪些两种可能的用途？

Answer (English):

Although most systems only distinguish between user and kernel modes, some CPUs have supported multiple modes. Multiple modes could be used to provide a finer-grained security policy. For example, rather than distinguishing between just user and kernel mode, you could distinguish between different types of user mode. Perhaps users belonging to the same group could execute each other's code. The machine would go into a specified mode when one of these users was running code. When the machine was in this mode, a member of the group could run code belonging to anyone else in the group.

Another possibility would be to provide different distinctions within kernel code. For example, a specific mode could allow USB device drivers to run. This would mean that USB devices could be serviced without having to switch to kernel mode, thereby essentially allowing USB device drivers to run in a quasi-user/kernel mode.

答案 (中文):

尽管大多数系统只区分用户和内核模式，但一些CPU支持多种模式。多模式可用于提供更细粒度的安全策略。例如，可以区分不同类型的用户模式，而不仅仅是用户和内核模式。也许属于同一组的用户可以执行彼此的代码。当这些用户之一运行代码时，机器进入指定模式。当机器处于此模式时，组成员可以运行组中任何其他人的代码。

另一种可能性是在内核代码中提供不同的区分。例如，特定模式可以允许USB设备驱动程序运行。这意味着USB设备可以在不切换到内核模式的情况下得到服务，从而基本上允许USB设备驱动程序在准用户/内核模式下运行。

1.9 Timers could be used to compute the current time. Provide a short description of how this could be accomplished.

中文：定时器可用于计算当前时间。简要描述如何实现这一点。

Answer (English):

A program could use the following approach to compute the current time using timer interrupts. The program could set a timer for some time in the future and go to sleep. When awakened by the interrupt, it could update its local state, which it uses to keep track of the number of interrupts it has received thus far. It could then repeat this process of continually setting timer interrupts and updating its local state when the interrupts are actually raised. The current time is base time plus the number of interrupts multiplies the time of interrupt.

答案 (中文):

程序可以使用以下方法利用定时器中断计算当前时间。程序可以将定时器设置为未来的某个时间并进入睡眠。当被中断唤醒时，它可以更新本地状态，用于跟踪到目前为止已接收的中断数量。然后它可以重复这个过程，持续设置定时器中断并在中断实际触发时更新本地状态。

1.10 Give two reasons why caches are useful. What problems do they solve? What problems do they cause? If a cache can be made as large as the device for which it is caching (for instance, a cache as large as a disk), why not make it that large and eliminate the device?

中文：给出缓存有用的两个原因。它们解决了什么问题？它们会引起什么问题？如果缓存可以做得和它所缓存的设备一样大（例如，缓存和磁盘一样大），为什么不把它做得那么大并消除该设备呢？

Answer (English):

Caches are useful when two or more components need to exchange data, and the components perform transfers at differing speeds. Caches solve the transfer problem by providing a buffer of intermediate speed between the components. If the fast device finds the data it needs in the

cache, it need not wait for the slower device. The data in the cache must be kept consistent with the data in the components. If a component has... (truncated) but main points: **caches solve speed mismatch** but **cause consistency issues and are expensive**. Making cache as large as device is **not economical**; also device provides **non-volatile storage that cache cannot replace**.

答案 (中文):

缓存有用是因为它们解决速度不匹配问题，减少等待时间。它们引起一致性问题（缓存与主存数据不一致）。缓存成本高，不能无限做大，而且设备本身提供持久存储，缓存通常易失，不能替代设备。

1.11 Distinguish between the client-server and peer-to-peer models of distributed systems.

中文：区分分布式系统的客户端-服务器模型和对等模型。

Answer (English):

The **client-server model** firmly distinguishes the roles of the client and server. Under this model, **the client requests services that are provided by the server**. The **peer-to-peer model** **doesn't have such strict roles**. In fact, **all nodes in the system are considered peers and thus may act as either clients or servers—or both**. A node may request a service from another peer, or the node may in fact provide such a service to other peers in the system.

For example, consider a system of nodes that share cooking recipes. Under the client-server model, all recipes are stored with the server. If a client wishes to access a recipe, it must request the recipe from the specified server. Using the peer-to-peer model, a peer node could ask other peer nodes for the specified recipe. The node (or perhaps nodes) with the requested recipe could provide it to the requesting node. Notice how each peer may act as both a client (it may request recipes) and as a server (it may provide recipes).

答案 (中文):

客户端-服务器模型明确区分客户端和服务端角色。在该模型下，客户端请求服务器提供的服务。对等模型没有如此严格的角色。实际上，系统中的所有节点都被视为对等体，因此可以充当客户端或服务器，或两者兼而有之。一个节点可以向另一个对等体请求服务，或者该节点实际上可以向系统中的其他对等体提供此类服务。

例如，考虑一个共享烹饪食谱的节点系统。在客户端-服务器模型下，所有食谱都存储在服务器上。如果客户端希望访问食谱，它必须向指定服务器请求食谱。使用对等模型，一个对等节点可以向其他对等节点询问指定食谱。拥有请求食谱的节点（或多个节点）可以将其提供给请求节点。注意每个对等体如何既充当客户端（请求食谱）又充当服务器（提供食谱）。

Chapter 2: Software processes (system calls and OS structure)

2.1 What is the purpose of system calls?

中文：系统调用的目的是什么？

Answer (English):

System calls allow user-level processes to request services of the operating system.

答案（中文）：

系统调用允许用户级进程请求操作系统的服务。

2.2 What is the purpose of the command interpreter? Why is it usually separate from the kernel?

中文：命令解释器的目的是什么？为什么它通常与内核分离？

Answer (English):

It reads commands from the user or from a file of commands and executes them, usually by turning them into one or more system calls. It is usually not part of the kernel because the command interpreter is subject to changes.

答案（中文）：

它从用户或命令文件读取命令并执行它们，通常通过将它们转换为一个或多个系统调用来实现。它通常不是内核的一部分，因为命令解释器易于变化。

2.3 What system calls have to be executed by a command interpreter or shell in order to start a new process on a UNIX system?

中文：在UNIX系统上，命令解释器或shell需要执行哪些系统调用来启动一个新进程？

Answer (English):

A `fork()` system call and an `exec()` system call need to be performed to start a new process. The `fork()` call clones the currently executing process, while the `exec()` call overlays a new process based on a different executable over the calling process.

答案（中文）：

需要执行 `fork()` 系统调用和 `exec()` 系统调用来启动新进程。`fork()` 调用克隆当前执行的进程，而 `exec()` 调用基于不同的可执行文件覆盖调用进程，以创建一个新进程。

2.4 What is the purpose of system programs?

中文：系统程序的目的是什么？

Answer (English):

System programs can be thought of as bundles of useful system calls. They provide basic functionality to users so that users do not need to write their own programs to solve common problems.

答案（中文）：

系统程序可以被看作是有用系统调用的集合。它们为用户提供基本功能，使用户无需编写自己的程序来解决常见问题。

2.5 What is the main advantage of the layered approach to system design? What are the disadvantages of the layered approach?

中文：分层系统设计的主要优点是什么？分层方法的缺点是什么？

Answer (English):

As in all cases of modular design, designing an operating system in a modular way has several advantages. The system is easier to debug and modify because changes affect only limited sections of the system rather than the whole. Disadvantages include performance overhead due to layer crossings, and the need for careful definition of layers.

答案（中文）：

与所有模块化设计一样，以模块化方式设计操作系统有几个优点。系统更容易调试和修改，因为更改只影响系统的有限部分，而不是整个系统。缺点包括层间交叉带来的性能开销，以及需要仔细定义各层。

2.6 List five services provided by an operating system, and explain how each creates convenience for users. In which cases would it be impossible for user-level programs to provide these services? Explain your answer.

中文：列出操作系统提供的五种服务，并解释每种服务如何为用户提供便利。在哪些情况下用户级程序无法提供这些服务？解释你的答案。

Answer (English):

The five services are:

- Program execution.** The operating system loads the contents (or sections) of a file into memory and begins its execution. A user-level program could not be trusted to properly allocate CPU time.
- I/O operations.** It is necessary to communicate with disks, tapes, and other devices at a very

low level. The user need only specify the device and the operation to perform on it, and the system converts that request into device- or controller-specific commands. User-level programs cannot be trusted to access only devices they should have access to and to access them only when they are otherwise unused.

c. File-system manipulation. There are many details in file creation, deletion, allocation, and naming that users should not have to perform. Blocks of disk space are used by files and must be tracked. Deleting a file requires removing the name file information and freeing the allocated blocks. Protections must also be checked to assure proper file access. User programs could neither ensure adherence to protection methods nor be trusted to allocate only free blocks and deallocate blocks on file deletion.

d. Communications. Message passing between systems requires messages to be turned into packets of information, sent to the network controller, transmitted across a communications medium, and reassembled by the destination system. Packet ordering and data correction must take place. Again, user programs might not coordinate access to the network device, or they might receive packets destined for other processes.

e. Error detection. Error detection occurs at both the hardware and software levels. At the hardware level, all data transfers must be inspected to ensure that data have not been corrupted in transit. All data on media must be checked to be sure they have not changed since they were written to the media. At the software level, media must be checked for data consistency—for instance, whether the number of allocated and unallocated blocks of storage match the total number on the device. There, errors are frequently process-independent, so there must be a global program (the operating system) that handles them.

答案（中文）：

五种服务是：

a. 程序执行。操作系统将文件内容（或部分）加载到内存并开始执行。用户级程序不能被信任来正确分配CPU时间。

b. I/O操作。需要以非常低的级别与磁盘、磁带和其他设备通信。用户只需指定设备和要执行的操作，系统将该请求转换为设备或控制器特定的命令。用户级程序不能被信任只访问它们应该访问的设备，并且只在设备空闲时访问它们。

c. 文件系统操作。文件创建、删除、分配和命名有许多细节，用户不必执行。磁盘空间块被文件使用，必须跟踪。删除文件需要移除文件名信息并释放已分配的块。还必须检查保护以确保正确的文件访问。用户程序既不能确保遵守保护方法，也不能被信任只分配空闲块并在文件删除时释放块。

d. 通信。系统间的消息传递需要将消息转换为信息包，发送到网络控制器，通过通信介质传输，并由目标系统重新组装。数据包排序和数据校正必须进行。同样，用户程序可能无法协调对网络设备的访问，或者可能接收到发往其他进程的数据包。

e. 错误检测。错误检测发生在硬件和软件级别。在硬件级别，必须检查所有数据传输以确保数据在传输过程中没有损坏。必须检查介质上的所有数据以确保它们自写入介质以来没有改变。在软件级别，必须检查介质的数据一致性——例如，已分配和未分配的存储块数量是否与设备上的总数匹配。这些错误通常是进程无关的，因此必须有一个全局程序（操作系统）来处理它们。

2.7 Why do some systems store the operating system in firmware, while others store it on disk?

中文：为什么有些系统将操作系统存储在固件中，而另一些将其存储在磁盘上？

Answer (English):

For certain devices, such as embedded systems, a disk with a file system may not be available for the device. In this situation, the operating system must be stored in firmware.

答案（中文）：

对于某些设备，如嵌入式系统，可能没有带文件系统的磁盘可用。在这种情况下，操作系统必须存储在固件中。

2.8 How could a system be designed to allow a choice of operating systems from which to boot? What would the bootstrap program need to do?

中文：如何设计一个系统以允许从多个操作系统中选择启动？引导程序需要做什么？

Answer (English):

Consider a system that would like to run both Windows and three different distributions of Linux (for example, RedHat, Debian, and Ubuntu). Each operating system will be stored on disk. During system boot, a special program (which we will call the boot manager) will determine which operating system to boot into. This means that rather than initially booting to an operating system, the boot manager will first run during system startup. It is this boot manager that is responsible for determining which system to boot into. Typically, boot managers must be stored at certain locations on the hard disk to be recognized during system startup. Boot managers often provide the user with a selection of systems to boot into; boot managers are also typically designed to boot into a default operating system if no choice is selected by the user.

答案（中文）：

考虑一个希望运行Windows和三种不同Linux发行版（例如RedHat、Debian和Ubuntu）的系统。每个操作系统都将存储在磁盘上。在系统启动期间，一个特殊程序（我们称之为启动管理器）将决定启动哪个操作系统。这意味着启动管理器将在系统启动期间首先运行，而不是直接启动到操作系统。正是这个启动管理器负责决定启动哪个系统。通常，启动管理器必须存储在硬盘的特定位置才能在系统启动时被识别。启动管理器通常为用户提供要启动的系统的选择；如果用户未选择，启动管理器通常也被设计为启动默认操作系统。

Chapter 3: Processes

3.1 Using the program shown in Figure 3.30, explain what the output will be at LINE A.

中文：运行图3.30中的程序，解释A行会输出什么？

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int value = 5;

int main()
{
    pid_t pid;

    pid = fork();

    if (pid == 0) { /* child process */
        value += 15;
        return 0;
    }
    else if (pid > 0) { /* parent process */
        wait(NULL);
        printf("PARENT: value = %d",value); /* LINE A */
        return 0;
    }
}
```

Figure 3.30 What output will be at Line A?

Answer: The result is still 5, as the child updates its copy of value. When control returns to the parent, its value remains at 5

3.2 Including the initial parent process, how many processes are created by the program shown in Figure 3.31?

中文：包括初始父进程，图3.31中的程序创建了多少个进程？

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    /* fork a child process */
    fork();

    /* fork another child process */
    fork();

    /* and fork another */
    fork();

    return 0;
}
```

Figure 3.31 How many processes are created?

Answer (English):

Eight processes are created.

答案 (中文):

创建了8个进程。

3.3 Original versions of Apple's mobile iOS operating system provided no means of concurrent processing. Discuss three major complications that concurrent processing adds to an operating system.

中文：Apple移动iOS操作系统的原始版本不提供并发处理手段。讨论并发处理给操作系统增加的三个主要复杂性。

Answer (English):

- The CPU scheduler must be aware of the different concurrent processes and must choose an appropriate algorithm that schedules the concurrent processes.
- Concurrent processes may need to communicate with one another, and the operating system must therefore develop one or more methods for providing interprocess communication.
- Because mobile devices often have limited memory, a process that manages memory poorly will have an overall negative impact on other concurrent processes. The operating system must therefore manage memory to support multiple concurrent processes.

答案 (中文):

- a. CPU调度器必须了解不同的并发进程，并必须选择适当的算法来调度并发进程。
 - b. 并发进程可能需要相互通信，因此操作系统必须开发一种或多种提供进程间通信的方法。
 - c. 由于移动设备通常内存有限，内存管理不佳的进程将对其他并发进程产生整体负面影响。因此操作系统必须管理内存以支持多个并发进程。
-

3.4 Some computer systems provide multiple register sets. Describe what happens when a context switch occurs if the new context is already loaded into one of the register sets. What happens if the new context is in memory rather than in a register set and all the register sets are in use?

中文：一些计算机系统提供多个寄存器组。描述如果新上下文已经加载到其中一个寄存器组中，发生上下文切换时会发生什么。如果新上下文在内存中而不是在寄存器组中，并且所有寄存器组都在使用中，会发生什么？

Answer (English):

The CPU current-register-set pointer is changed to point to the set containing the new context, which takes very little time. If the context is in memory, one of the contexts in a register set must be chosen and be moved to memory, and the new context must be loaded from memory into the set. This process takes a little more time than on systems with one set of registers, depending on how a replacement victim is selected.

答案 (中文):

CPU当前寄存器组指针被更改为指向包含新上下文的组，这只需要很少的时间。如果上下文在内存中，必须选择一个寄存器组中的上下文并移入内存，然后从内存加载新上下文到该组。这个过程比只有一组寄存器的系统需要更多时间，具体取决于替换受害者的选择方式。

3.5 When a process creates a new process using the fork() operation, which of the following states is shared between the parent process and the child process?

- a. Stack
- b. Heap
- c. Shared memory segments

中文：当进程使用fork()操作创建新进程时，以下哪种状态在父进程和子进程之间共享？

- a. 栈
- b. 堆
- c. 共享内存段

Answer (English):

Only the shared memory segments are shared between the parent process and the newly

forked child process. Copies of the stack and the heap are made for the newly created process.

答案 (中文):

只有共享内存段在父进程和新创建的子进程之间共享。栈和堆的副本会为新创建的进程创建。

3.6 Consider the "exactly once" semantic with respect to the RPC mechanism. Does the algorithm for implementing this semantic execute correctly even if the ACK message sent back to the client is lost due to a network problem? Describe the sequence of messages, and discuss whether "exactly once" is still preserved.

中文：考虑RPC机制中的“恰好一次”语义。实现该语义的算法即使由于网络问题导致返回给客户端的ACK消息丢失，也能正确执行吗？描述消息序列，并讨论“恰好一次”是否仍然得到保证。

Answer (English):

The "exactly once" semantics ensure that a remote procedure will be executed exactly once and only once. The general algorithm for ensuring this combines an acknowledgment (ACK) scheme combined with timestamps (or some other incremental counter that allows the server to distinguish between duplicate messages).

The general strategy is for the client to send the RPC to the server along with a timestamp. The client will also start a timeout clock. The client will then wait for one of two occurrences: (1) it will receive an ACK from the server indicating that the remote procedure was performed, or (2) it will time out. If the client times out, it assumes the server was unable to perform the remote procedure, so the client invokes the RPC a second time, sending a later timestamp. The client may not receive the ACK for one of two reasons: (1) the original RPC was never received by the server, or (2) the RPC was correctly received—and performed—by the server but the ACK was lost. In situation (1), the use of ACKs allows the server ultimately to receive and perform the RPC. In situation (2), the server will receive a duplicate RPC, and it will use the timestamp to identify it as a duplicate so as not to perform the RPC a second time. It is important to note that the server must send a second ACK back to the client to inform the client the RPC has been performed.

答案 (中文):

“恰好一次”语义确保远程过程将恰好执行一次且仅一次。确保这一点的通用算法结合了确认 (ACK) 方案和时间戳 (或其他允许服务器区分重复消息的增量计数器)。

一般策略是客户端向服务器发送RPC并附带时间戳。客户端还启动一个超时时钟。然后客户端等待以下两种情况之一：(1) 收到服务器的ACK，表示远程过程已执行，或 (2) 超时。如果客户端超时，它假设服务器无法执行远程过程，因此客户端第二次调用RPC，发送较晚的时间戳。客户端可能因以下两个原因之一未收到ACK：(1) 原始RPC从未被服务器接收，或 (2) RPC已被正确接收和执行，但ACK丢失。在情况(1)中，使用ACK最终允许服务器接收并执行RPC。在情况(2)中，服务器将收到重复的RPC，并使用时间戳将其识别为重复，以免第二次执行该RPC。重要的是服务器必须向客户端发送第二个ACK，告知客户端RPC已执行。

3.7 Assume that a distributed system is susceptible to server failure. What mechanisms would be required to guarantee the "exactly once" semantic for execution of RPCs?

中文：假设一个分布式系统容易发生服务器故障。需要什么机制来保证RPC执行的“恰好一次”语义？

Answer (English):

The server should keep track in stable storage (such as a disk log) of information regarding what RPC operations were received, whether they were successfully performed, and the results associated with the operations. When a server crash takes place and an RPC message is received, the server can check whether the RPC has been previously performed and therefore guarantee "exactly once" semantics for the execution of RPCs.

答案（中文）：

服务器应在稳定存储（如磁盘日志）中跟踪有关收到的RPC操作、是否成功执行以及相关结果的信息。当服务器崩溃并收到RPC消息时，服务器可以检查该RPC是否先前已执行，从而保证RPC执行的“恰好一次”语义。

Chapter 4: Threads & Concurrency

4.1 Provide three programming examples in which multithreading provides better performance than a single-threaded solution.

中文：给出三个多线程编程优于单线程解决方案的例子。

Answer (English):

- A web server that services each request in a separate thread
- A parallelized application such as matrix multiplication where various parts of the matrix can be worked on in parallel
- An interactive GUI program such as a debugger where one thread is used to monitor user input, another thread represents the running application, and a third thread monitors performance

答案（中文）：

- 一个Web服务器，每个请求单独使用一个线程来处理
 - 一个并行化的应用程序，如矩阵乘法，矩阵的不同部分可以并行计算
 - 一个交互式GUI程序，比如调试器，其中一个线程用于监控用户输入，另一个线程表示正在运行的应用程序，第三个线程监控性能
-

4.2 Using Amdahl's Law, calculate the speedup gain of an application that has a 60 percent parallel component for (a) two processing cores and (b) four processing cores.

中文：使用阿姆达尔定律，计算一个并行部分为60%的应用程序在（a）两个处理核心和（b）四个处理核心下的加速比。

Amdahl's law:

$$\text{Speedup} = \frac{1}{(1 - P) + \frac{P}{S}}$$

where P is the percentage of concurrency and S is the speedup times of concurrency, generally is the number of processing cores.

Answer (English):

- a. With two processing cores we get a speedup of 1.42 times.
- b. With four processing cores, we get a speedup of 1.82 times.

答案（中文）：

- a. 使用两个处理核心时，加速比为1.42倍。
 - b. 使用四个处理核心时，加速比为1.82倍。
-

4.3 Does the multithreaded web server described in Section 4.1 exhibit task or data parallelism?

中文：第4.1节中描述的多线程Web服务器展示的是任务并行还是数据并行？

Answer (English):

Data parallelism. Each thread is performing the same task, but on different data.

答案（中文）：

数据并行。每个线程执行相同的任务，但处理的数据不同。

4.4 What are two differences between user-level threads and kernel-level threads? Under what circumstances is one type better than the other?

中文：用户级线程和内核级线程之间有哪些两个区别？在什么情况下一种类型优于另一种？

Answer (English):

- a. User-level threads are unknown by the kernel, whereas the kernel is aware of kernel threads.
- b. User-level threads are managed by a thread library in user space without kernel intervention (fast creation and switching), while kernel-level threads are managed directly by the operating system kernel (slower but can take advantage of multiple CPUs).

User-level threads are better when frequent thread creation and switching are needed and when the application does not require true parallelism across multiple CPUs. Kernel-level threads are better when the application needs to utilize multiple processor cores and when threads may block on I/O without blocking the entire process.

答案（中文）：

- a. 用户级线程对内核不可见，而内核级线程对内核是可见的。
 - b. 用户级线程由用户空间的线程库管理，无需内核干预（创建和切换速度快），而内核级线程由操作系统内核直接管理（速度较慢，但可以利用多CPU并行）。
- 用户级线程在需要频繁创建和切换线程，且应用不需要跨多CPU真正并行时更好。内核级线程在应用需要利用多处理器核心，以及线程可能因I/O阻塞而不阻塞整个进程时更好。
-

4.5 Describe the actions taken by a kernel to context-switch between kernel-level threads.

中文：描述内核在内核级线程之间进行上下文切换时所采取的动作。

Answer (English):

Context switching between kernel threads typically requires saving the value of the CPU registers from the thread being switched out and restoring the CPU registers of the new thread being scheduled.

答案（中文）：

内核线程之间的上下文切换通常需要保存被切换的线程的CPU寄存器值，并恢复被调度到的新线程的CPU寄存器值。

4.6 What resources are used when a thread is created? How do they differ from those used when a process is created?

中文：创建线程时使用哪些资源？它们与创建进程时使用的资源有何不同？

Answer (English):

Because a thread is smaller than a process, thread creation typically uses fewer resources than process creation. Creating a process requires allocating a process control block (PCB), a rather large data structure. The PCB includes a memory map, a list of open files, and environment variables. Allocating and managing the memory map is typically the most time-consuming activity. Creating either a user thread or a kernel thread involves allocating a small data structure to hold a register set, stack, and priority.

答案（中文）：

因为线程比进程小，线程创建通常使用的资源少于进程创建。创建进程需要分配进程控制块

(PCB)，这是一个相当大的数据结构。PCB包含内存映射、打开文件列表和环境变量。分配和管理内存映射通常是最耗时的活动。创建用户线程或内核线程都只涉及分配一个小的数据结构来保存寄存器组、栈和优先级。

4.7 Assume that an operating system maps user-level threads to the kernel using the many-to-many model and that the mapping is done through LWPs. Furthermore, the system allows developers to create real-time threads for use in real-time systems. Is it necessary to bind a real-time thread to an LWP? Explain.

中文：假设一个操作系统使用多对多模型将用户级线程映射到内核，且映射是通过LWP完成的。此外，系统允许开发者创建实时线程用于实时系统。是否有必要将实时线程绑定到一个LWP？解释原因。

LWP: Light Weighted Process, intermediate data structure between kernel and user-level threads

Answer (English):

Yes. Timing is crucial to real-time applications. If a thread is marked as real-time but is not bound to an LWP, the thread may have to wait to be attached to an LWP before running. Consider a situation in which a real-time thread is running (is attached to an LWP) and then proceeds to block (must perform I/O, has been preempted by a higher-priority real-time thread, is waiting for a mutual exclusion lock, etc.). While the real-time thread is blocked, the LWP it was attached to is assigned to another thread. When the real-time thread has been scheduled to run again, it must first wait to be attached to an LWP. By binding an LWP to a real-time thread, you are ensuring that the thread will be able to run with minimal delay once it is scheduled.

答案（中文）：

是的。时序对实时应用至关重要。如果一个线程被标记为实时线程但不绑定到LWP，那么该线程在运行前可能必须先等待被附加到一个LWP。考虑一种情况：一个实时线程正在运行（已附加到LWP），然后它进入阻塞状态（必须执行I/O、被更高优先级的实时线程抢占、等待互斥锁等）。当该实时线程被阻塞时，它所附加的LWP被分配给另一个线程。当该实时线程被调度再次运行时，它必须首先等待被附加到LWP。通过将LWP绑定到实时线程，你可以确保该线程一旦被调度就能以最小的延迟运行。

Chapter 5: CPU Scheduling

5.1 A CPU-scheduling algorithm determines an order for the execution of its scheduled processes. Given n processes to be scheduled on one processor, how many different schedules are possible? Give a formula in terms of n .

中文：CPU调度算法决定了其调度进程的执行顺序。给定 n 个进程在一个处理器上调度，可能有多少种不同的调度顺序？给出用 n 表示的公式。

Answer (English):

$n!$ (n factorial).

答案（中文）：

$n!$ (n 的阶乘)。

5.2 Explain the difference between preemptive and nonpreemptive scheduling.

中文：解释抢占式调度和非抢占式调度的区别。

Answer (English):

Preemptive scheduling allows a process to be interrupted in the midst of its execution, taking the CPU away and allocating it to another process. Nonpreemptive scheduling ensures that a process relinquishes control of the CPU only when it finishes with its current CPU burst.

答案（中文）：

抢占式调度允许进程在执行过程中被中断，将CPU分配给另一个进程。非抢占式调度确保进程仅在完成当前CPU突发后才释放CPU控制权。

5.3 Suppose that the following processes arrive for execution at the times indicated. Each process will run for the amount of time listed. In answering the questions, use nonpreemptive scheduling, and base all decisions on the information you have at the time the decision must be made.

Process	Arrival	Burst Time
P_1	0.0	8
P_2	0.4	4
P_3	1.0	1

- What is the average turnaround time for FCFS?
- What is the average turnaround time for SJF?
- Compute average turnaround time if CPU is left idle for the first 1 unit and then SJF (future-knowledge scheduling).

Non-preemptive:

- FCFS(first come first serve): schedule by arriving order
- SJF(shortest job first): Choose the process that has arrived and has the shortest burst time.
- Priority scheduling: choose the process with higher priority

Preemptive:

- RR: allocate time slice to each process and execute in order
- SRTF(shortest remaining time first): preemptive version of SJF, each choose the process with shortest remaining time.

中文：假设以下进程在所示时间到达执行。每个进程将运行列出的时间量。回答问题使用非抢占调度，并根据决策时拥有的信息作出决策。

进程：P1到达0突发8；P2到达0.4突发4；P3到达1.0突发1。

- 使用FCFS调度，平均周转时间是多少？
- 使用SJF调度，平均周转时间是多少？
- 如果CPU在第一个1个时间单位内空闲然后使用SJF（未来知识调度），平均周转时间是多少？

Answer (English):

- 10.53
- 9.53
- 6.86

(Remember that turnaround time is finishing time minus arrival time.)

答案（中文）：

- 10.53
- 9.53
- 6.86

(记住周转时间是完成时间减去到达时间。)

5.4 Consider the following set of processes with burst times and priorities. (All arrive at 0).

Process	Burst Time	Priority
P_1	2	2
P_2	1	1
P_3	8	4
P_4	4	2
P_5	5	3

- Draw Gantt charts for FCFS, SJF, nonpreemptive priority (larger number higher), RR (quantum=2).
- Turnaround times for each algorithm.
- Waiting times.
- Which algorithm minimizes average waiting time?
 - turnaround time = finishing time - arriving time
 - waiting time = turnaround time - burst time

中文：考虑以下进程集合... (均于0到达)。

- 画出FCFS、SJF、非抢占优先级（数大优先级高）、RR（时间片=2）的甘特图。
- 每种算法的周转时间。
- 等待时间。
- 哪种算法平均等待时间最小？

Answer (English): (from PDF tables)

b. Turnaround:

FCFS: P1=2, P2=3, P3=11, P4=15, P5=20

SJF: P1=3, P2=7, P3=12, P4=13, P5=19

Priority: P1=1, P2=3, P3=11, P4=20, P5=13

RR: P1=2, P2=5, P3=11, P4=20, P5=13

c. Waiting (turnaround - burst):

FCFS: 0,2,3,11,15

SJF: 1,2,4,9,14

Priority: 0,2,3,16,8

RR: 0,3,3,16,8

d. SJF has the shortest average waiting time.

答案 (中文):

b. 周转时间:

FCFS: P1=2, P2=3, P3=11, P4=15, P5=20

SJF: P1=3, P2=7, P3=12, P4=13, P5=19

优先级: P1=1, P2=3, P3=11, P4=20, P5=13

RR: P1=2, P2=5, P3=11, P4=20, P5=13

c. 等待时间 (周转-突发):

FCFS: 0,2,3,11,15

SJF: 1,2,4,9,14

优先级: 0,2,3,16,8

RR: 0,3,3,16,8

d. SJF的平均等待时间最短。

5.5 (Round-robin with priorities and idle task) – Gantt, turnaround, waiting, CPU utilization.

中文：（带优先级和空闲任务的循环调度）——甘特图、周转、等待、CPU利用率。

Process	Priority	Burst	Arrival
P_1	40	20	0
P_2	30	25	25
P_3	30	25	30
P_4	35	15	60
P_5	5	10	100
P_6	10	10	105

Each process is assigned a numerical priority, with a higher number indicating a higher relative priority. In addition to the processes listed above, the system also has an idle task (which consumes no CPU resources and is identified as P_{idle}). This task has priority 0 and is scheduled whenever the system has no other available processes to run. The length of a time quantum is 10 units. If a process is preempted by a higher-priority process, the preempted process is placed at the end of the queue.

- First execute process with high priority
- RR for same priority

Answer (English): (from PDF)

- Gantt: P1, idle, P2, P3, P2, P3, P4, P2, P3, idle, P5, P6 (with times).
- Turnaround: P1=20, P2=55, P3=60, P4=15, P5=20, P6=10.
- Waiting: P1=0, P2=40, P3=35, P4=0, P5=10, P6=0.
- CPU utilization = $105/120 = 87.5\%$.

答案（中文）：

- 甘特图：P1, 空闲, P2, P3, P2, P3, P4, P2, P3, 空闲, P5, P6（带时间）。
- 周转：P1=20, P2=55, P3=60, P4=15, P5=20, P6=10。
- 等待：P1=0, P2=40, P3=35, P4=0, P5=10, P6=0。
- CPU利用率 = $105/120 = 87.5\%$ 。

5.6 What advantage is there in having different time-quantum sizes at different levels of a multilevel queueing system?

中文：在多级队列系统的不同级别具有不同时间片大小有什么优点？

Answer (English):

Processes that need **more frequent servicing**—for instance, interactive processes such as editors—can be in a queue with a **small time quantum**. Processes with no need for frequent servicing can be in a queue with a larger quantum, requiring fewer context switches to complete the processing and thus **making more efficient use of the computer**.

答案 (中文):

需要更频繁服务的进程——例如，交互式进程如编辑器——可以放在小时间片的队列中。不需要频繁服务的进程可以放在较大时间片的队列中，需要较少的上下文切换来完成处理，从而更有效地利用计算机。

5.7 (Relationship between algorithm sets)

中文：（算法集合之间的关系）—— a. 优先级和SJF； b. 多级反馈队列和FCFS； c. 优先级和FCFS； d. RR和SJF。

Many CPU-scheduling algorithms are parameterized. For example, the RR algorithm requires a parameter to indicate the time slice. Multilevel feedback queues require parameters to define the number of queues, the scheduling algorithms for each queue, the criteria used to move processes between queues, and so on. These algorithms are thus really sets of algorithms (for example, the set of RR algorithms for all time slices, and so on). One set of algorithms may include another (for example, the FCFS algorithm is the RR algorithm with an infinite time quantum). What (if any) relation holds between the following pairs of algorithm sets?

- a. Priority and SJF
- b. Multilevel feedback queues and FCFS
- c. Priority and FCFS
- d. RR and SJF

Answer (English):

- a. SJF is a special case of priority scheduling where priority is inverse of next CPU burst. **The shortest job has the highest priority**
- b. FCFS is a special case of multilevel feedback queues with a single queue and no feedback. **The lowest level of MLFQ is FCFS.**
- c. **FCFS gives the highest priority to the job that has been in existence the longest.**
- d. RR and SJF are unrelated; RR uses time slicing, SJF uses shortest burst. None.

答案 (中文):

- a. SJF是优先级调度的一种特殊情况，其中优先级是下次CPU突发的倒数。
- b. FCFS是只有一个队列且无反馈的多级反馈队列的特例。
- c. FCFS是所有优先级相等（或无抢占）的优先级调度。
- d. RR和SJF无关；RR使用时间片，SJF使用最短突发。

5.8 Suppose that a CPU scheduling algorithm favors those processes that have used the least processor time in the recent past. Why will this algorithm favor I/O-bound programs and yet not permanently starve CPU-bound programs?

中文：假设CPU调度算法偏爱那些最近过去使用处理器时间最少的进程。为什么这种算法会偏爱I/O密集型程序，但又不会永久饿死CPU密集型程序？

Answer (English):

It will favor the I/O-bound programs because of the relatively short CPU bursts requested by them; however, the CPU-bound programs will not starve, because the I/O-bound programs will relinquish the CPU relatively often to do their I/O.

答案（中文）：

它会偏爱I/O密集型程序，因为它们请求的CPU突发相对较短；然而，CPU密集型程序不会饿死，因为I/O密集型程序会相对频繁地放弃CPU来进行I/O操作。

5.9 Distinguish between PCS and SCS scheduling.

中文：区分PCS和SCS调度。

Answer (English):

PCS scheduling is local to the process. It is how the thread library schedules threads onto available LWPs. SCS scheduling is used when the operating system schedules kernel threads. On systems using either the many-to-one or the many-to-many model, the two scheduling models are fundamentally different. On systems using the one-to-one model, PCS and SCS are the same.

答案（中文）：

PCS调度是进程本地的。它是指线程库如何将线程调度到可用的LWP上。SCS调度用于操作系统调度内核线程。在使用多对一或多对多模型的系统上，这两种调度模型根本不同。在使用一对一模型的系统上，PCS和SCS相同。

5.10 The traditional UNIX scheduler recalculates process priorities once per second using $\text{Priority} = (\text{recent CPU usage} / 2) + \text{base}$, where $\text{base}=60$. Assume recent CPU usage for $P1=40$, $P2=18$, $P3=10$. What will be the new priorities? Based on this, does the scheduler raise or lower the relative priority of a CPU-bound process?

中文：传统的UNIX调度器每秒使用 $\text{Priority} = (\text{最近CPU使用量}/2) + \text{base}$ ($\text{base}=60$) 重新计算进程优先级。假设P1的最近CPU使用量为40，P2为18，P3为10。新的优先级是什么？基于此，调度器是提高还是降低CPU密集型进程的相对优先级？

Answer (English):

Priorities: P1=80, P2=69, P3=65. The scheduler lowers the relative priority of CPU-bound processes.

答案 (中文):

优先级: P1=80, P2=69, P3=65。调度器降低CPU密集型进程的相对优先级。

Chapter 6: Synchronization

6.1 In Section 6.4, we mentioned that disabling interrupts frequently can affect the system's clock. Explain why this can occur and how such effects can be minimized.

中文: 在6.4节中, 我们提到频繁禁用中断可能影响系统时钟。解释为什么会发生这种情况以及如何最小化这种影响。

Answer (English):

The system clock is updated at every clock interrupt. If interrupts were disabled—particularly for a long period of time—the system clock could easily lose the correct time. The system clock is also used for scheduling purposes. For example, the time quantum for a process is expressed as a number of clock ticks. At every clock interrupt, the scheduler determines if the time quantum for the currently running process has expired. If clock interrupts were disabled, the scheduler could not accurately assign time quanta. This effect can be minimized by disabling clock interrupts for only very short periods.

答案 (中文):

系统时钟在每个时钟中断时更新。如果中断被禁用——尤其是长时间禁用——系统时钟很容易丢失正确时间。系统时钟也用于调度目的。例如, 进程的时间片以时钟滴答数表示。在每个时钟中断, 调度器确定当前运行进程的时间片是否已到期。如果时钟中断被禁用, 调度器无法准确分配时间片。通过仅禁用时钟中断很短的时间可以最小化这种影响。

6.2 What is the meaning of the term busy waiting? What other kinds of waiting are there in an operating system? Can busy waiting be avoided altogether? Explain your answer.

中文: 术语“忙等待”的含义是什么? 操作系统中还有其他什么类型的等待? 忙等待能被完全避免吗? 解释你的答案。

Answer (English):

Busy waiting means that a process is waiting for a condition to be satisfied in a tight loop without relinquishing the processor. One strategy to avoid busy waiting temporarily puts the waiting process to sleep and awakens it when the appropriate program state is reached, but

this solution incurs the overhead associated with putting the process to sleep and later waking it up.

Blocking waiting is another type of waiting. The process will relinquish the processor automatically and go to sleep. Then the process will go to the waiting queue. CPU will choose another ready process.

I/O waiting is a specific subset of blocking waiting. Waiting for completion of I/O operations on external devices.

Cannot be avoided altogether. Blocking waiting will introduce the overhead of context switching which exceeds the cost of simply spinning a few CPU cycles.

答案（中文）：

忙等待是指进程在紧密循环中等待条件满足而不放弃处理器。避免忙等待的一种策略是暂时将等待进程置于睡眠状态，并在达到适当程序状态时唤醒它，但这种解决方案会带来将进程放入睡眠和随后唤醒的开销。

6.3 Explain why spinlocks are not appropriate for single-processor systems yet are often used in multiprocessor systems.

中文：解释为什么自旋锁不适合单处理器系统，但经常用于多处理器系统。

Answer (English):

Spinlocks are not appropriate for single-processor systems because the condition that would break a process out of the spinlock can be obtained only by executing a different process. If the process is not relinquishing the processor, other processes do not get the opportunity to set the program condition required for the first process to make progress. In a multiprocessor system, other processes execute on other processors and therefore can modify the program state in order to release the first process from the spinlock.

答案（中文）：

自旋锁不适合单处理器系统，因为使进程退出自旋锁的条件只能通过执行另一个进程来获得。如果该进程不释放处理器，其他进程就没有机会设置第一个进程进展所需的程序条件。在多处理器系统中，其他进程在其他处理器上执行，因此可以修改程序状态以释放第一个进程自旋锁。

6.4 Show that, if the wait() and signal() semaphore operations are not executed atomically, then mutual exclusion may be violated.

中文：证明如果 wait() 和 signal() 信号量操作不是原子执行的，则可能违反互斥。

Answer (English):

A wait() operation atomically decrements the value associated with a semaphore. If two wait() operations are executed on a semaphore when its value is 1 and the operations are not performed atomically, then both operations might decrement the semaphore value, thereby violating mutual exclusion.

答案 (中文):

wait() 操作原子地减少与信号量关联的值。如果当信号量值为1时两个 wait() 操作在同一信号量上执行，且操作不是原子执行的，则两个操作都可能减少信号量值，从而违反互斥。

6.5 Illustrate how a binary semaphore can be used to implement mutual exclusion among n processes.

中文：说明如何使用二进制信号量实现 n 个进程之间的互斥。

Answer (English):

The n processes share a semaphore, mutex, initialized to 1. Each process P_i is organized as follows:

```
do {
    wait(mutex);

    /*critical section*/

    signal(mutex);

    /*remainder section*/
} while (true);
```

答案 (中文):

这 n 个进程共享一个信号量 mutex，初始化为1。每个进程 P_i 组织如下：

do { wait(mutex); / 临界区 / signal(mutex); / 剩余区 / } while (true);

6.6 Race conditions are possible in many computer systems. Consider a banking system that maintains an account balance with two functions: deposit(amount) and withdraw(amount). These two functions are passed the amount that is to be deposited or withdrawn from the bank account balance. Assume that a husband and wife share a bank account. Concurrently, the husband calls the withdraw() function, and the wife calls deposit(). Describe how a race condition is possible and what might be done to prevent the race condition from occurring.

中文：许多计算机系统中可能出现竞态条件。考虑一个银行系统，维护账户余额，有两个函数：存款(amount)和取款(amount)。... 描述竞态条件如何可能发生，以及可以采取什么措施防止竞态条件发生。

Answer (English):

Assume balance is \$250. Husband calls withdraw(\$50) and wife calls deposit(\$100). Correct value should be \$300. If transactions are interleaved, the local value for husband becomes \$200 but before commit, deposit(100) updates shared balance to \$300. Then switch back to husband, and shared balance is set to \$200, an incorrect value. Prevention: use mutual exclusion (lock) around the deposit/withdraw operations.

答案（中文）：

假设余额为250美元。丈夫调用取款(50)而妻子调用存款(100)。正确值应为300美元。如果事务交错，丈夫的本地值变为200，但在提交之前，存款(100)将共享余额更新为300。然后切换回丈夫，共享余额被设置为200，这是错误值。防止：在存款/取款操作周围使用互斥（锁）。

Chapter 7: Synchronization Examples

7.1 Explain why Windows and Linux implement multiple locking mechanisms. Describe the circumstances under which they use spinlocks, mutex locks, semaphores, and condition variables. In each case, explain why the mechanism is needed.

中文：解释为什么Windows和Linux实现多种锁定机制。描述在什么情况下它们使用自旋锁、互斥锁、信号量和条件变量。每种情况下解释为什么需要该机制。

Answer (English):

These operating systems provide different locking mechanisms depending on the application developers' needs. Spinlocks are useful for multiprocessor systems where a thread can run in a busy loop (for a short period of time) rather than incurring the overhead of being put in a sleep queue. Mutexes are useful for locking resources. Solaris 2 uses adaptive mutexes, meaning that the mutex is implemented with a spinlock on multiprocessor machines. Semaphores and condition variables are more appropriate tools for synchronization when a resource must be held for a long period of time, since spinning is inefficient for a long duration.

答案（中文）：

这些操作系统根据应用开发者的需求提供不同的锁定机制。自旋锁在多处理器系统中很有用，线程可以在忙循环中运行（短时间），而不是承受进入睡眠队列的开销。互斥锁用于锁定资源。Solaris 2使用自适应互斥锁，即在多处理器机器上互斥锁用自旋锁实现。信号量和条件变量更适合资源必须长时间持有的同步，因为长时间自旋效率低下。

7.2 Windows provides a lightweight synchronization tool called slim reader-writer locks.

... Explain the benefits of providing such a synchronization tool.

中文：Windows提供了一种轻量级同步工具，称为轻量级读写锁（slim reader-writer locks）。... 解释提供这种同步工具的好处。

Answer (English):

Simplicity. If reader-writer locks provide fairness or favor readers or writers, they involve more overhead. Providing such a simple synchronization mechanism makes access to the lock fast. Use of this lock may be most appropriate for situations where reader-writer locks are needed, but quickly acquiring and releasing them is similarly important.

答案（中文）：

简单性。如果读写锁提供公平性或偏向读者或写者，它们会带来更多开销。提供这种简单的同步机制使锁访问快速。这种锁可能最适用于需要读写锁但快速获取和释放同样重要的情况。

7.4 Describe how deadlock is possible with the dining-philosophers problem.

中文：描述哲学家就餐问题中死锁如何可能发生。

Answer (English):

If all philosophers simultaneously pick up their left forks, when they turn to pick up their right forks they will realize they are unavailable, and will block while waiting for it to become available. This blocking while waiting for a resource to become available is a deadlocked situation.

答案（中文）：

如果所有哲学家同时拿起左边的叉子，当他们转而拿起右边的叉子时，会发现它们不可用，并在等待可用时阻塞。这种等待资源可用时的阻塞就是死锁情况。

7.5 Explain the difference between signaled and non-signaled states with Windows dispatcher objects.

中文：解释Windows调度器对象的已通知状态和未通知状态的区别。

Answer (English):

An object that is in the signaled state is available, and a thread will not block when it tries to acquire it. When the lock is acquired, it is in the non-signaled state. When the lock is released, it transitions back to the signaled state.

答案（中文）：

处于已通知状态的对象可用，线程尝试获取时不会阻塞。当锁被获取时，它处于未通知状态。当锁被释放时，它转换回已通知状态。

7.6 Assume val is an atomic integer in a Linux system. What is the value of val after the following operations have been completed?

```
atomic_set(&val, 10);
atomic_sub(8, &val);
atomic_inc(&val);
atomic_inc(&val);
atomic_add(6, &val);
atomic_sub(3, &val);
```

中文：假设val是Linux系统中的原子整数。执行以下操作后val的值是多少？

Answer (English):

Final value = $10 - 8 + 1 + 1 + 6 - 3 = 7$

答案（中文）：

最终值 = $10 - 8 + 1 + 1 + 6 - 3 = 7$

Chapter 8: Deadlocks

8.1 List three examples of deadlocks that are not related to a computer-system environment.

中文：列出三个与计算机系统环境无关的死锁例子。

Answer (English):

- Two cars crossing a single-lane bridge from opposite directions.
- A person going down a ladder while another person is climbing up the ladder.
- Two trains traveling toward each other on the same track.

答案（中文）：

- 两辆汽车从相反方向穿过单车道桥。
 - 一个人下梯子而另一个人爬梯子。
 - 两列火车在同一条轨道上相向行驶。
-

8.2 Suppose that a system is in an unsafe state. Show that it is possible for the threads to complete their execution without entering a deadlocked state.

中文：假设系统处于不安全状态。证明线程有可能完成执行而不进入死锁状态。

Answer (English):

An unsafe state may not necessarily lead to deadlock, it just means that we cannot guarantee that deadlock will not occur. Thus, it is possible that a system in an unsafe state may still allow all processes to complete without deadlock occurring. Consider the situation where a system has twelve resources allocated among processes P0, P1, and P2. (Example omitted but conclusion: it is possible that a process may release resources before requesting any further resources, thereby avoiding deadlock.)

答案（中文）：

不安全状态不一定导致死锁，它只是意味着我们不能保证死锁不会发生。因此，处于不安全状态的系统仍可能允许所有进程完成而不发生死锁。考虑一个系统有十二个资源分配给进程P0、P1和P2的情况。有可能进程在请求更多资源之前释放资源，从而避免死锁。

8.3 Consider the following snapshot of a system: (table with Allocation, Max, Available)

Answer using Banker's algorithm.

	Allocation	Max	Available
	A B C D	A B C D	A B C D
T0	0 0 1 2	0 0 1 2	1 5 2 0
T1	1 0 0 0	1 7 5 0	
T2	1 3 5 4	2 3 5 6	
T3	0 6 3 2	0 6 5 2	
T4	0 0 1 4	0 6 5 6	

- What is the content of the matrix Need?
- Is the system in a safe state?
- If a request from thread T1 arrives for (0,4,2,0), can the request be granted immediately?

中文：考虑以下系统快照... 使用银行家算法回答。

- Need矩阵的内容是什么？
- 系统是否处于安全状态？
- 如果线程T1请求 (0,4,2,0)，能否立即授予？

Answer (English):

- Need: T0=(0,0,0,0), T1=(0,7,5,0), T2=(1,0,0,2), T3=(0,0,2,0), T4=(0,6,4,2).

- b. Safe, Available=(1,5,2,0), can run T0 or T3.
- c. Yes, can be granted immediately.

答案 (中文):

- a. Need: T0=(0,0,0,0), T1=(0,7,5,0), T2=(1,0,0,2), T3=(0,0,2,0), T4=(0,6,4,2)。
- b. 安全, Available=(1,5,2,0), 可运行T0或T3。
- c. 可以, 立即授予。

8.4 A possible method for preventing deadlocks is to have a single, higher-order resource that must be requested before any other resource. ... Compare this scheme with the circular-wait scheme.

中文: 一种可能的死锁预防方法是要求在任何其他资源之前必须请求一个单一的更高级资源。 ... 将该方案与循环等待方案进行比较。

(the access of objects are managed by the higher-order resource as a lock)

Answer (English):

This is probably **not a good solution** because **it yields too large a scope**. It is better to define a locking policy with as narrow a scope as possible. The **circular wait approach is a reasonable approach** to avoiding deadlock, and **does not increase the scope of holding a lock**.

答案 (中文):

这可能不是一个好的解决方案, 因为它产生太大的范围。最好定义范围尽可能窄的锁定策略。循环等待方法是避免死锁的合理方法, 并且不会增加持有锁的范围。

8.5 Prove that the safety algorithm presented in Section 8.6.3 requires an order of $m \times n^2$ operations.

中文: 证明8.6.3节中给出的安全性算法需要 $m \times n^2$ 次操作量级。

Answer (English):

```
for (int i = 0; i < n; i++){
    // first find a thread that can finish
    for (int j = 0; j < n; j++){
        if (!finish[j]){
            boolean temp = true;
            for (int k = 0; k < m; k++){
                if (need[j][k] > work[k]){
                    temp = false;
                }
            }
        }
    }
}
```

```

        if (temp){
            // if this thread can finish
            finish[j] = true;
            for (int x = 0; x < m; x++){
                work[x] += work[j][x];
            }
        }
    }
}

```

The nested loops give n^2 and inner loops over m , so $O(m \times n^2)$.

答案 (中文):

嵌套循环给出 n^2 ，内部循环遍历 m ，因此复杂度为 $O(m \times n^2)$ 。

8.6 Consider a computer system that runs 5,000 jobs per month ... (pros and cons of installing deadlock-avoidance).

- Arguments for installing.
- Arguments against.

中文：考虑一个月运行5000个作业的计算机系统... (安装死锁避免算法的利弊)。

- 支持安装的论点。
- 反对安装的论点。

Answer (English):

- For: Deadlock would **never occur** and all 5000 jobs could **still run**.
- Against: **increases average execution time** and turnaround, **deadlock occurs infrequently**.

答案 (中文):

- 支持：避免昂贵的重新运行，提高可靠性，防止死锁。
- 反对：增加平均执行时间和周转时间，如果死锁很少发生可能不值得。

8.7 Can a system detect that some of its threads are starving? If you answer "yes," explain how. If "no," explain how to deal with starvation.

中文：系统能否检测到某些线程饥饿？如果回答“是”，解释如何检测。如果“否”，解释如何处理饥饿。

Answer (English):

Starvation is **difficult to define**. One way is to **set a maximum waiting time T** ; if a process waits

longer than T, it is starved. Strategies include assigning resources to the longest-waiting process or using aging.

答案（中文）：

饥饿难以定义。一种方法是设置最大等待时间T；如果进程等待超过T，则认为饥饿。策略包括将资源分配给等待最长的进程或使用老化。

8.8 Consider the following resource-allocation policy ... (preemptive). Can deadlock occur? Can indefinite blocking occur?

中文：考虑以下资源分配策略（抢占式）。死锁会发生吗？无限期阻塞会发生吗？

Answer (English):

a. Deadlock cannot occur because preemption exists.

b. Yes, A process may never acquire all the resources it needs if they are continuously preempted by a series of requests from other processes.

答案（中文）：

a. 死锁不会发生，因为存在抢占。违反了“不可抢占”条件。

b. 是的。如果一个进程的资源被其他进程的一系列请求连续抢占，它可能永远无法获得所需的所有资源。。

8.9 Consider snapshot (table) and answer using Banker's algorithm for two Available states.

	Allocation	Max
	A B C D	A B C D
T0	3 0 1 4	5 1 1 7
T1	2 2 1 0	3 2 1 1
T2	3 1 2 1	3 3 2 1
T3	0 5 1 0	4 6 1 2
T4	4 2 1 2	6 3 2 5

中文：考虑快照并针对两个Available状态使用银行家算法回答。

a. available = 0 3 0 1

b. available = 1 0 0 2

Answer (English):

- a. Not safe.
- b. Safe, order: P1, P2, P3, then P0, P4.

答案 (中文):

- a. 不安全。
 - b. 安全, 顺序: P1, P2, P3, 然后 P0, P4。
-

8.10 Suppose that you have coded the safety algorithm ... Can you use it for deadlock detection by redefining $Max_i = Waiting_i + Allocation_i$?

中文: 假设你已经编写了安全性算法... 能否通过重新定义Max来用于死锁检测?

Answer (English):

Yes. $Max = Waiting + Allocation$. So replace Need with Waiting.

答案 (中文):

可以。 $Max = Waiting + Allocation$, 所以用Waiting代替Need。

8.11 Is it possible to have a deadlock involving only one single-threaded process? Explain.

中文: 单个单线程进程可能发​​生死锁吗? 解释。

Answer (English):

No, because hold-and-wait condition requires at least two processes.

答案 (中文):

不可能, 因为“持有并等待”条件至少需要两个进程。

Chapter 9: Memory Management

9.1 Name two differences between logical and physical addresses.

中文: 说出逻辑地址和物理地址的两个区别。

Answer (English):

A logical address does not refer to an actual physical address; it refers to an abstract address. Physical address refers to actual memory location. Logical address is generated by CPU and translated by MMU; physical address is output of MMU.

memory management unit maps logical address into physical address through page table.

答案 (中文):

逻辑地址不指向实际物理地址；它指向抽象地址。物理地址指向实际内存位置。逻辑地址由CPU生成，由MMU转换；物理地址是MMU的输出。

9.2 Why are page sizes always powers of 2?

中文：为什么页大小总是2的幂？

Answer (English):

Because **paging splits address into page and offset bits**. Using power-of-2 allows **efficient bit splitting rather than arithmetic division**.

答案 (中文):

因为分页将地址拆分为页号和偏移量位。使用2的幂允许高效的位拆分，而不是算术除法。

9.3 Consider a **system with two base-limit register pairs for code and data. Discuss **advantages and disadvantages**.**

中文：考虑一个具有代码和数据两对基址-限长寄存器的系统。讨论优缺点。

Answer (English):

Advantage: **efficient sharing of read-only code** among users, protection of code from modification. Disadvantage: increased **hardware complexity**, **limited flexibility** if code and data need to be intermingled.

答案 (中文):

优点：可在用户间有效共享只读代码，保护代码不被修改。缺点：增加硬件复杂性，如果代码和数据需要混合则灵活性受限。

9.4 Consider a logical address space of 64 pages of 1,024 words each, mapped onto physical memory of 32 frames. How many bits in logical address? Physical?

中文：考虑一个逻辑地址空间有64页，每页1024字，映射到32帧的物理内存。逻辑地址有多少位？物理地址有多少位？

Physical address is obtained through **replacing** page number by frame number. The offset(words) keeps.

Answer (English):

Logical: 64 pages = 2^6 , 1024 words = 2^{10} → total 16 bits. Physical: 32 frames = 2^5 , offset 10 bits → total 15 bits.

答案 (中文):

逻辑: 64页= 2^6 , 1024字= 2^{10} → 共16位。物理: 32帧= 2^5 , 偏移10位 → 共15位。

9.5 What is the effect of allowing two entries in a page table to point to the same page frame? How could this be used to copy memory efficiently? What effect does updating one page have on the other?

中文: 允许页表中两个表项指向同一页框会产生什么效果? 这如何用于高效复制内存? 更新一页对另一页有什么影响?

Answer (English):

Allows sharing of code/data. Copying can be done by pointing multiple page tables to same frames. Updating one page changes the other, so shared data must be read-only or carefully synchronized.

答案 (中文):

允许共享代码/数据。可以通过让多个页表指向相同帧来实现复制。更新一页会改变另一页, 因此共享数据必须是只读的或仔细同步。

9.6 Given six memory partitions (300KB, 600KB, 350KB, 200KB, 750KB, 125KB in order) how would first-fit, best-fit, worst-fit place processes of given sizes 115, 500, 358, 200, 375 KB in order?

中文: 给定六个内存分区... 首次适应、最佳适应、最差适应如何放置指定大小的进程?

- first-fit: find in order and place in the first partition. Fastest and leave piece in low address
- best-fit: find all partitions and place in the smallest fit partition. Maintain large block of space but generate many small pieces.
- worst-fit: find all partitions and place in the largest partition. The pieces are large that may be used again. But may cause large process cannot be placed.

Answer (English): (detailed from PDF)

a. First fit:

- b. 115 KB is put in 300-KB partition, leaving 185 KB, 600 KB, 350 KB, 200 KB, 750 KB, 125 KB
- c. 500 KB is put in 600-KB partition, leaving 185 KB, 100 KB, 350 KB, 200 KB, 750 KB, 125 KB
- d. 358 KB is put in 750-KB partition, leaving 185 KB, 100 KB, 350 KB, 200 KB, 392 KB, 125 KB

- e. 200 KB is put in 350-KB partition, leaving 185 KB, 100 KB, 150 KB, 200 KB, 392 KB, 125 KB
- f. 375 KB is put in 392-KB partition, leaving 185 KB, 100 KB, 150 KB, 200 KB, 17 KB, 125 KB
- g. Best fit:
- h. 115 KB is put in 125-KB partition, leaving 300 KB, 600 KB, 350 KB, 200 KB, 750 KB, 10 KB
- i. 500 KB is put in 600-KB partition, leaving 300 KB, 100 KB, 350 KB, 200 KB, 750 KB, 10 KB
- j. 358 KB is put in 750-KB partition, leaving 300 KB, 100 KB, 350 KB, 200 KB, 392 KB, 10 KB
- k. 200 KB is put in 200-KB partition, leaving 300 KB, 100 KB, 350 KB, 0 KB, 392 KB, 10 KB
- l. 375 KB is put in 392-KB partition, leaving 300 KB, 100 KB, 350 KB, 0 KB, 17 KB, 10 KB

答案 (中文):

- 首次适应: 115→300, 500→600, 358→750, 200→350, 375→392 (留下碎片)。
- 最佳适应: 115→125, 500→600, 358→750, 200→200, 375→392。
- 最差适应: 115→750, 500→635, 358→600, 200→350, 375等待。

9.7 Assuming a 1-KB page size, what are page numbers and offsets for decimal addresses: 3085, 42095, 215201, 650000, 2000001?

中文: 假设页大小为1KB, 十进制地址的页号和偏移量是什么?

logical address = page number * page size + offset

page number = logical address / page size

offset = logical address % offset

Answer (English):

- a. page=3, offset=13
- b. page=41, offset=111
- c. page=210, offset=161
- d. page=634, offset=784
- e. page=1953, offset=129

答案 (中文):

- a. 页号=3, 偏移=13
- b. 页号=41, 偏移=111
- c. 页号=210, 偏移=161
- d. 页号=634, 偏移=784
- e. 页号=1953, 偏移=129

9.8 The BTV operating system has a 21-bit virtual address, 16-bit physical address, 2-KB page size. How many entries in single-level page table?

中文：BTV操作系统有21位虚拟地址，16位物理地址，2KB页大小。单级页表有多少项？

- inverted page table: maps physical to logical

Answer (English):

Page size 2KB = 2^{11} , so virtual page number bits = $21 - 11 = 10 \rightarrow 2^{10} = 1024$ entries.

inverted: $16 - 11 = 5$. $2^5 = 32$.

答案 (中文):

页大小2KB = 2^{11} ，所以虚拟页号位数 = $21 - 11 = 10 \rightarrow 2^{10} = 1024$ 项。

9.9 Consider logical address space of 256 pages with 4-KB page size, mapped to 64 frames. How many bits in logical and physical address?

中文：考虑逻辑地址空间有256页，每页4KB，映射到64帧。逻辑和物理地址各多少位？

Answer (English):

Logical: $256 = 2^8$, 4KB = $2^{12} \rightarrow$ total 20 bits. Physical: $64 = 2^6$, offset 12 $\rightarrow$ 18 bits.

答案 (中文):

逻辑：20位，物理：18位。

9.10 Consider a system with 32-bit logical address, 4-KB page size, supports 512 MB physical memory. How many entries in conventional and inverted page table?

中文：考虑32位逻辑地址，4KB页大小，支持512MB物理内存。常规页表和倒置页表各有多少项？

Answer (English):

Conventional: 2^{20} entries. Inverted: $512\text{MB}/4\text{KB} = 128\text{K}$ entries.

答案 (中文):

常规： 2^{20} 项。倒置：128K项。

Chapter 10: Virtual Memory

10.1

English: Under what circumstances do page faults occur? Describe the actions taken by the operating system when a page fault occurs.

Answer: A page fault occurs when an access to a page that has not been brought into main

memory takes place. The operating system verifies the memory access, aborting the program if it is invalid. If it is valid, a free frame is located and I/O is requested to read the needed page into the free frame. Upon completion of I/O, the process table and page table are updated, and the instruction is restarted.

中文： 在什么情况下会发生缺页故障？描述缺页故障发生时操作系统采取的动作。

答案： 当访问尚未调入主存的页面时会发生缺页故障。操作系统验证内存访问，若无效则终止程序；若有效，则找一个空闲帧，并请求I/O将所需页面读入该帧。I/O完成后，更新进程表和页表，并重新执行该指令。

10.2

English: Assume that you have a page-reference string for a process with m frames (initially all empty). The page-reference string has length p , and n distinct page numbers occur in it. Answer these questions for any page-replacement algorithms: a. What is a lower bound on the number of page faults? b. What is an upper bound on the number of page faults?

Answer: a. n ; b. p .

中文： 假设有一个进程的页引用串，分配了 m 个帧（初始均为空）。页引用串长度为 p ，其中包含 n 个不同的页号。对于任意页置换算法：a. 缺页次数的下界是多少？b. 缺页次数的上界是多少？

答案： a. n ; b. p 。

10.3

English: Consider the following page-replacement algorithms. Rank these algorithms on a five-point scale from "bad" to "perfect" according to their page-fault rate. Separate those algorithms that suffer from Belady's anomaly from those that do not. a. LRU replacement b. FIFO replacement c. Optimal replacement d. Second-chance replacement

Answer: (The original answer is not explicitly given in the text, but typical ranking: Optimal – perfect (no Belady); LRU – good (no Belady); Second-chance – fair (suffer Belady); FIFO – bad (suffers Belady)).

中文： 考虑以下页置换算法。根据缺页率从“差”到“完美”按五级评分。区分哪些算法受Belady异常影响，哪些不受。a. LRU置换 b. FIFO置换 c. 最优置换 d. 二次机会置换。

答案： （PDF中未给出明确答案，但典型评价：最优——完美（无Belady）；LRU——良好（无Belady）；二次机会——一般（有Belady）；FIFO——差（有Belady）。）

10.4

English: An operating system supports a paged virtual memory. The central processor has a cycle time of 1 microsecond. It costs an additional 1 microsecond to access a page other than the current one. Pages have 1,000 words, and the paging device is a drum that rotates at 3,000

revolutions per minute and transfers 1 million words per second. The following statistical measurements were obtained from the system: One percent of all instructions executed accessed a page other than the current page. Of the instructions that accessed another page, 80 percent accessed a page already in memory. When a new page was required, the replaced page was modified 50 percent of the time. Calculate the effective instruction time on this system, assuming that the system is running one process only and that the processor is idle during drum transfers.

Answer: effective access time = $0.99 \times (1\mu\text{sec} + 0.008 \times (2\mu\text{sec})) + 0.002 \times (10,000\mu\text{sec} + 1,000\mu\text{sec}) + 0.001 \times (10,000\mu\text{sec} + 1,000\mu\text{sec}) = (0.99 + 0.016 + 22.0 + 11.0) \mu\text{sec} = 34.0 \mu\text{sec}$.

中文：某操作系统支持请求分页虚拟内存。CPU周期为1微秒。访问非当前页额外耗时1微秒。每页1000字，分页设备为鼓，转速3000转/分，传输率1百万字/秒。测得统计：1%的指令访问非当前页；其中80%访问已在内存的页；需要新页时，被置换页50%被修改。计算有效指令执行时间（假设单进程，鼓传输时CPU空闲）。

答案：有效访问时间 = $0.99 \times (1\mu\text{s} + 0.008 \times (2\mu\text{s})) + 0.002 \times (10,000\mu\text{s} + 1,000\mu\text{s}) + 0.001 \times (10,000\mu\text{s} + 1,000\mu\text{s}) = 34.0 \mu\text{s}$ 。

10.5

English: Consider the page table for a system with 12-bit virtual and physical addresses and 256-byte pages. (Table given) The list of free page frames is D, E, F (D at head). Convert the following virtual addresses to physical addresses in hexadecimal: 9EF, 111, 700, 0FF.

Answer: 9EF → 0EF; 111 → 211; 700 → D00; 0FF → EFF.

中文：考虑一个12位虚拟地址和物理地址、256字节页面的系统，页表如表所示。空闲页帧列表为D, E, F（D在头）。将以下虚拟地址转换为物理地址（十六进制）：9EF, 111, 700, 0FF。

答案：9EF → 0EF; 111 → 211; 700 → D00; 0FF → EFF。

10.6

English: Discuss the hardware functions required to support demand paging.

Answer: For every memory-access operation, the page table must be consulted to check whether the corresponding page is resident and whether the program has read or write privileges. These checks must be performed in hardware. A TLB could serve as a cache and improve performance.

中文：讨论支持请求分页所需的硬件功能。

答案：每次内存访问操作都必须查阅页表，检查对应页是否驻留以及程序是否有读写权限。这些检查必须由硬件完成。TLB可作为缓存提高查找性能。

10.7

English: Consider the two-dimensional array A: `int A[100][100]`; where `A[0][0]` is at location 200 in a paged memory system with pages of size 200. A small process manipulates the matrix and resides in page 0 (locations 0 to 199). For three page frames, how many page faults are generated by the following loops? Use LRU, assume frame 1 contains the process, other two empty. a. `for (j=0; j<100; j++) for (i=0; i<100; i++) A[i][j]=0;` b. `for (i=0; i<100; i++) for (j=0; j<100; j++) A[i][j]=0;`

Answer: a. 5,000; b. 50.

中文: 考虑二维数组A: `int A[100][100]`; `A[0][0]`位于地址200, 页大小为200。一个小进程操作该矩阵, 驻留在页0 (地址0-199)。分配三个页帧, 帧1给进程, 另两个初始为空。使用LRU, 以下循环会产生多少次缺页? a. 按列优先; b. 按行优先。

答案: a. 5,000; b. 50。

10.8

English: Consider the page reference string: 1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6. How many page faults for LRU, FIFO, Optimal with 1 to 7 frames? (Table given)

Answer: (Table provided in PDF)

Number of frames: 1 2 3 4 5 6 7

LRU: 20 18 15 10 8 7 7

FIFO: 20 18 16 14 10 10 7

Optimal: 20 15 11 8 7 7 7

中文: 考虑页引用串: 1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6。分别用LRU、FIFO、最优置换, 帧数为1至7时各产生多少次缺页?

答案: (见上表) 帧数1~7, LRU: 20,18,15,10,8,7,7; FIFO: 20,18,16,14,10,10,7; 最优: 20,15,11,8,7,7,7。

10.9

English: Consider reference string: 7,2,3,1,2,5,3,4,6,7,7,1,0,5,4,6,2,3,0,1. Assuming demand paging with three frames, how many page faults for LRU, FIFO, Optimal?

Answer: LRU: 18; FIFO: 17; Optimal: 13.

中文: 参考串: 7,2,3,1,2,5,3,4,6,7,7,1,0,5,4,6,2,3,0,1。假设请求分页, 三帧, 分别用LRU、FIFO、最优置换的缺页次数?

答案: LRU: 18; FIFO: 17; 最优: 13。

10.10

English: Suppose that you want to use a **paging algorithm that requires a reference bit**, but

hardware does not provide one. Sketch how you could simulate a reference bit, or explain why not. If possible, calculate the cost.

Answer: You can use the valid/invalid bit to simulate. Initially set to invalid. On first reference, a trap occurs; the OS sets a software bit to 1 and resets valid/invalid to valid.

中文: 假设要使用需要引用位的页置换算法, 但硬件未提供。请简述如何模拟引用位, 或解释为何不能。若可能, 计算开销。

答案: 可利用有效/无效位模拟。初始设为无效。首次访问产生陷阱, OS将软件位置1, 并将有效/无效位设为有效。

10.11

English: You have devised a new page-replacement algorithm that you think may be optimal. In some contorted test cases, Belady's anomaly occurs. Is the new algorithm optimal? Explain.

Answer: No. An optimal algorithm will not suffer from Belady's anomaly because it evicts the page that will not be used for the longest time. Belady's anomaly occurs when a page needed soon is evicted.

中文: 你设计了一个可能最优的新页置换算法, 但在某些测试用例中出现了Belady异常。该算法是否最优? 解释。

答案: 不是。最优算法不会出现Belady异常, 因为它会淘汰将来最久不被使用的页。Belady异常意味着淘汰了即将需要的页。

10.12

English: Segmentation is similar to paging but uses variable-sized "pages." Define two segment-replacement algorithms, one based on FIFO and one on LRU. Remember segments not same size; chosen segment may be too small. Consider relocation possible or not.

Mainly consider the victim and the situation that the size of victim is not large enough.

Answer: (a) FIFO: Find first segment large enough. If relocation not possible and no single segment fits, combine contiguous segments. If relocation possible, rearrange memory to make contiguous space. (b) LRU: Similar but choose least recently used segment that fits or can be combined.

中文: 分段类似于分页, 但使用可变大小的“页”。定义两种段置换算法, 分别基于FIFO和LRU。注意段大小不等, 被选中的段可能太小。考虑段是否可重定位。

答案: FIFO: 找到第一个足够大的段; 若不可重定位且无单个段够大, 则合并连续的段; 若可重定位, 则重新安排内存使空间连续。LRU类似, 但选择最近最少使用且够大或可合并的段。

10.13

English: Consider a demand-paged system with degree of multiprogramming fixed at four. Utilization measured: a. CPU 13%, disk 97%; b. CPU 87%, disk 3%; c. CPU 13%, disk 3%. For each, what is happening? Can multiprogramming be increased to improve CPU utilization? Is paging helping?

degree of multiprogramming is the number of processes that are in the memory and compete for the CPU at the same time. It is used to improve the utilization of CPU.

Answer: a. Thrashing – cannot increase; paging not helping. b. CPU high – can increase multiprogramming; paging helping. c. CPU low, disk low – increase multiprogramming.

中文: 某请求分页系统，多道程序度固定为4。测得利用率：a. CPU 13%，磁盘97%；b. CPU 87%，磁盘3%；c. CPU 13%，磁盘3%。每种情况发生了什么？能否增加多道度提高CPU利用率？分页是否有帮助？

答案: a. 抖动——不能增加多道度，分页无帮助；b. CPU利用率高——可增加多道度，分页有帮助；c. CPU和磁盘都低——应增加多道度。

10.14

English: We have an OS for a machine that uses base and limit registers, but we modified it to provide a page table. Can the page table be set up to simulate base and limit registers? How?

- base register stores the start address(base address) in physical memory of current process
- limit register stores largest scale of logical address of current process
- physical address = base address + logical address in early CPU. Process should be continuous in the physical memory.

Answer: Yes, if memory is allocated in fixed-size segments. The base of a segment can be entered into the first page of page table, and the valid/invalid bit used to indicate resident portion. Internal fragmentation may occur.

中文: 某机器原本使用基址和限长寄存器，现改为提供页表。页表能否模拟基址和限长寄存器？如何模拟？

答案: 可以，前提是内存按固定大小段分配。可将段的基址填入页表，并用有效/无效位指示驻留部分。可能存在内部碎片。

Chapter 11: Mass-Storage Structure

11.1

English: Is disk scheduling, other than FCFS, useful in a single-user environment? Explain.

Answer: In a single-user environment, the I/O queue is usually empty. FCFS is economical, but

LOOK is nearly as easy and gives better performance when multiple processes do concurrent I/O (e.g., web browser, paging, foreground app).

中文： 在单用户环境中，除FCFS外的磁盘调度是否有用？解释。

答案： 单用户环境I/O队列通常为0，FCFS经济，但LOOK几乎同样简单，且在多进程并发I/O（如浏览器、分页、前台应用）时性能更好。

11.2

English: Explain why SSTF scheduling tends to favor middle cylinders over the innermost and outermost cylinders.

Answer: The center of the disk has the smallest average distance to all other tracks. The head tends to move away from edges. If head not at center, new requests are more likely in the direction of the center.

中文： 解释为什么SSTF调度倾向于中间柱面，而不是最内或最外柱面。

答案： 磁盘中心到所有其他磁道的平均距离最小。磁头趋向于远离边缘。若磁头不在中心，新请求更可能位于指向中心的方向。

11.3

English: Why is rotational latency usually not considered in disk scheduling? How would you modify SSTF, SCAN, and C-SCAN to include latency optimization?

Answer: Most disks do not export rotational position to host; even if they did, imprecision and scheduler variability would make it obsolete. Requests are usually given as logical block numbers, not physical locations. To include latency, the scheduler would need real-time rotational position and could choose among requests with similar cylinder distances the one with shortest rotational delay.

中文： 为什么磁盘调度中通常不考虑旋转延迟？如何修改SSTF、SCAN和C-SCAN以包含延迟优化？

答案： 多数磁盘不向主机提供旋转位置信息；即使提供，不精确性和调度器可变性也会使其过时。请求通常以逻辑块号给出。要包含延迟，调度器需实时旋转位置，并在柱面距离相近的请求中选择旋转延迟最短的。

11.4

English: Why is it important to balance file-system I/O among disks and controllers in a multitasking environment?

Answer: A system performs at the speed of its slowest bottleneck. Disks/controllers are often bottlenecks. Balancing I/O avoids overwhelming any single disk or controller.

中文： 为什么在多任务环境中平衡文件系统I/O在磁盘和控制器之间很重要？

答案：系统性能受限于最慢瓶颈。磁盘/控制器常为瓶颈。平衡I/O可避免单个磁盘或控制器过载。

11.5

English: What are the tradeoffs involved in rereading code pages from the file system versus using swap space to store them?

Answer: Swap space transfers faster (tuned for performance). Using swap space may require startup time to copy pages there, and more swap space is needed if used for both code and data.

中文：从文件系统重读代码页与使用交换空间存储代码页之间有何权衡？

答案：交换空间传输更快（经过性能调优）。使用交换空间可能需要在进程启动时复制页面，且若同时用于代码和数据，需更多交换空间。

11.6

English: Is there any way to implement truly stable storage? Explain.

Answer: Truly stable storage would never lose data. Maintaining multiple copies helps, but a disaster could destroy all copies. Thus, truly stable storage is not possible.

中文：是否有方法实现真正稳定的存储？解释。

答案：真正稳定的存储永远不会丢失数据。维护多份副本有帮助，但灾难可能摧毁所有副本，因此不可能实现。

11.7

English: (Detailed calculation problem) Given latency and transfer rates for cache, memory, disk, tape. a. Compute effective transfer rate for disk for transfers of 512B, 8KB, 1MB, 16MB. b. Utilization. c. Smallest transfer size for 25% utilization for disk. d. Complete sentence: disk is random-access for transfers larger than bytes. e. Minimum transfer sizes for cache, memory, tape for acceptable utilization. f. When is tape random-access vs sequential?

Answer: a. 512B: 33.12 KB/sec; 8KB: 0.47 MB/sec; 1MB: 4.65 MB/sec; 16MB: 4.98 MB/sec. b. Utilizations: 0.64%, 9.4%, 93%, 99.6%. c. ~25 KB (0.025 MB). d. Disk is random-access for transfers larger than K bytes (where $K > \text{block size}$), and sequential for smaller. e. Cache: ~2.24 bytes; Memory: ~1.68 bytes; Tape: ~40 MB. f. Tape is sequential when restoring backup; random when accessing a variety of records.

中文：（详细计算题）给定缓存、内存、磁盘、磁带的延迟和传输率。a. 计算磁盘在传输 512B、8KB、1MB、16MB 时的有效传输率。b. 利用率。c. 磁盘达到 25% 利用率的最小传输大小。d. 完成句子：磁盘对于大于字节的传输是随机访问设备。e. 缓存、内存、磁带达到可接受利用率的最小传输大小。f. 磁带何时为随机访问，何时为顺序访问？

答案： a. 512B: 33.12 KB/s; 8KB: 0.47 MB/s; 1MB: 4.65 MB/s; 16MB: 4.98 MB/s。 b. 利用率分别为0.64%, 9.4%, 93%, 99.6%。 c. 约25 KB (0.025 MB)。 d. 磁盘对于大于K字节 (K>块大小) 的传输是随机访问, 对于更小的则是顺序访问。 e. 缓存约2.24字节; 内存约1.68字节; 磁带约40 MB。 f. 恢复备份时磁带为顺序访问; 访问任意记录时为随机访问。

For d and f,

- the difference of random-access device and sequential-access device is that effective transfer rate is closer to streaming transfer rate. If it is larger than utilization(25%), it is random access.

11.8

English: Could a RAID level 1 organization achieve better performance for read requests than RAID level 0 (nonredundant striping)? If so, how?

- RAID: redundant array of independent disks. Binding many physical disks together to form a logically store unit.
- RAID 0(striping): data is divided into chunks and stores separately on different disks. For specific chunk, it is on a specific physical disk
- RAID 1: store a copy on other disk(s). For specific chunk, controller can choose a disk whose head is closer to the chunk.
- RAID 0 performs for handle different requests concurrently while RAID 1 performs for single chunk.
- RAID 5(striping with parity): check code are distributed on different disks. It can accept one disk damage.
- RAID 6: two different algorithm to calculate check code. Allow two disks damage.

Answer: Yes. RAID 1 can choose which copy of a block to read based on current head position, optimizing performance by selecting the closer disk head.

中文： RAID 1组织在读请求方面能否比RAID 0（非冗余条带）获得更好性能？若能，如何实现？

答案： 能。RAID 1可根据当前磁头位置选择读取哪个副本，选择较近的磁头来优化性能。

11.9

English: Give three reasons to use HDDs as secondary storage.

Answer: 1. Largest random-access storage for the price (terabytes at low cost). 2. Many devices designed for HDDs provide wide compatibility. 3. HDDs maintain consistent read/write performance over lifetime, unlike NVM which degrades with fullness and age.

中文： 给出使用HDD作为辅助存储的三个理由。

答案： 1. 性价比最高的大容量随机访问存储（低成本的TB级）。2. 大量设备为HDD设计，兼容性广。3. HDD在整个生命周期内读写性能稳定，而NVM随填充和老化性能下降。

11.10

English: Give three reasons to use NVM devices as secondary storage.

Answer: 1. High-speed NVM (SSDs) are much faster than HDDs, greatly impacting system performance. 2. Use less power and are smaller, suitable for portable/battery devices. 3. No moving parts, so more reliable than HDDs.

中文： 给出使用NVM设备作为辅助存储的三个理由。

答案： 1. 高速NVM（如SSD）远快于HDD，显著影响系统性能。2. 功耗更低、体积更小，适合便携/电池设备。3. 无活动部件，比HDD更可靠。

Chapter 12: I/O Systems

12.1

English: State three advantages and three disadvantages of placing functionality in a device controller, rather than in the kernel.

Answer: Advantages: a. Bugs less likely to crash OS. b. Performance improved by dedicated hardware. c. Kernel simplified. Disadvantages: a. Bug fixes require firmware/hardware update. b. Algorithm improvements require hardware update. c. Embedded algorithms may conflict with application use, decreasing performance.

中文： 说出将功能放在设备控制器中而不是内核中的三个优点和三个缺点。

答案： 优点：a. 错误不易导致OS崩溃。b. 专用硬件提高性能。c. 内核简化。缺点：a. 修复错误需更新固件/硬件。b. 改进算法需更新硬件。c. 嵌入式算法可能与应用程序使用冲突，降低性能。

12.2

English: The example of handshaking used two bits (busy and command-ready). Is it possible with only one bit? If so, describe protocol; if not, explain why.

- handshaking: protocol by two bits. Busy bit maintained by controller while command-ready bit maintained by CPU.

Answer: Yes. Use the busy bit (or command-ready). When bit is off, controller idle. Host writes data and sets bit to signal operation ready. When controller finishes, it clears bit. Host then starts next operation.

This operation can complicate circuitry and increase the cost of the controller.

中文：握手示例使用了两个位（忙和命令就绪）。能否只用一个位实现？若能，描述协议；若不能，解释为什么。

答案：能。只使用忙位（或命令就绪位）。当位为0时控制器空闲。主机写入数据并置位表示操作就绪。控制器完成后清除该位，主机开始下一次操作。

12.3

English: Why might a system use interrupt-driven I/O for a single serial port and polling I/O for a terminal concentrator?

- Polling: CPU loops to check whether controller ready
- Interrupt: CPU do other operations until controller sends a interrupt.

Answer: Polling is efficient when I/O is frequent and short. A single serial port has infrequent I/O, so interrupts are better. A terminal concentrator has many short I/Os; interrupting each would cause high load, so polling is better.

中文：为什么系统可能对单个串口使用中断驱动I/O，而对终端集中器使用轮询I/O？

答案：轮询在I/O频繁且短时时更高效。单个串口I/O不频繁，适合中断。终端集中器有许多短I/O，每次中断会造成高负载，故轮询更好。

12.4

English: Polling can waste CPU cycles if the device is not ready. Describe a hybrid strategy combining polling, sleeping, and interrupts. For each strategy (pure polling, pure interrupts, hybrid), describe a computing environment where it is more efficient.

Answer: Hybrid: Poll for N cycles; if still busy, set interrupt and sleep. Best for very long or very short waits; inefficient if I/O completes at N+T (small T) due to overhead. Pure polling: best for very short wait times. Interrupts: best for known long wait times.

中文：若设备未就绪，轮询会浪费CPU周期。描述结合轮询、睡眠和中断的混合策略。对纯轮询、纯中断、混合策略，各描述一种更有效的计算环境。

答案：混合：轮询N次；若仍忙，则设置中断并睡眠。适用于极长或极短的等待；若在N+T（T很小）完成则因开销而不佳。纯轮询适合极短等待；纯中断适合已知长等待。

12.5

English: How does DMA increase system concurrency? How does it complicate hardware design?

- DMA: direct memory access. Allow external devices directly transfer data to memory without involvement of CPU. After CPU sends the information of data to be transferred, DMA controller masters buses and CPU can do other things until interrupt is sent by DMA controller.
- Cycle stealing: DMA controller occupies the bus, which causes CPU cannot use buses simultaneously.

Answer: DMA allows CPU to perform tasks while DMA transfers data via buses. Hardware design complicated because DMA controller must be integrated as bus master, and cycle stealing may be needed to share memory bus.

中文： DMA如何提高系统并发性？它如何使硬件设计复杂化？

答案： DMA让CPU在DMA通过总线传输数据时执行其他任务。硬件设计复杂因为DMA控制器必须作为总线主控集成，且可能需要周期窃取以共享内存总线。

12.6

English: Why is it important to scale up system-bus and device speeds as CPU speed increases?

Answer: To remove the system bottleneck when CPU speed is high, we need to increase all components to increase overall system performance rather than blindly increasing the performance of individual system components.

中文： 为什么随着CPU速度提高，提升系统总线和设备速度很重要？

12.7

English: Distinguish between a driver end and a stream module in a STREAMS operation.

- Streams operations: operates data when transfer it between user space and device.
- user spaces - stream head - stream module(s) - driver end - stream module

Answer: The driver end controls a physical device involved in the STREAMS operation. The stream module modifies the flow of data between the stream head (user interface) and the driver.

中文： 区分STREAMS操作中的驱动程序端和流模块。

答案： 驱动程序端控制参与STREAMS操作的物理设备。流模块修改流头部（用户接口）和驱动程序之间的数据流。

Chapter 13: File-System Interface

13.1

English: Some systems automatically delete all user files when a user logs off or job terminates, unless explicitly kept. Others keep all files unless explicitly deleted. Discuss relative merits.

Answer: Deleting unsaved files saves disk space and reduces clutter. Keeping all files gives better security against accidental loss.

中文: 有些系统在用户注销或作业终止时自动删除所有用户文件（除非明确保留），而另一些则保留所有文件除非明确删除。讨论各自的优点。

答案: 删除未保存文件节省磁盘空间、减少杂乱。保留所有文件可防止意外丢失。

13.2

English: Why do some systems keep track of file type, while others leave it to the user or don't implement multiple types? Which is "better"?

Answer: Some systems support different operations per type (e.g., ASCII vs database). Others leave interpretation to processes. Which is better depends on needs; for database-heavy systems, OS-supported types are efficient; for general-purpose, simpler types allow flexibility.

中文: 为什么有些系统记录文件类型，而另一些留给用户或根本不实现多种类型？哪种“更好”？

答案: 有些系统针对不同类型支持不同操作（如ASCII vs 数据库）。另一些将解释留给进程。哪种更好取决于需求；对数据库密集系统，OS支持类型更高效；对通用系统，简单类型更灵活。

13.3

English: Similarly, some systems support many structures for file data, others just a byte stream. Advantages and disadvantages?

Answer: Advantages of system support: individual applications are not required to provide the support; more efficiently than an application.

Disadvantages of system support: increases the size of system; applications that require file types may not be able to run on the system.

Advantages of byte stream: simplify the OS; allow applications to define file structures to avoid the situation that the system may not provide a file definition for a specific application.

中文: 类似地，有些系统支持多种文件数据结构，而另一些只支持字节流。优缺点？

13.4

English: Could you simulate a multilevel directory structure with a single-level directory using arbitrarily long names? If yes, how? Contrast with multilevel scheme. How would answer change if names limited to 7 characters?

Answer: Yes, use **delimiters** (e.g., "jim.java.F1") to indicate subdirectories. If limited to 7 chars, **cannot**; instead use a symbol table mapping long names to short names.

中文: 能否使用单级目录结构（允许任意长文件名）模拟多级目录？若能，如何？对比多级方案。若文件名限制为7个字符，答案如何变化？

答案: 能，使用分隔符（如“jim.java.F1”）表示子目录。若限制为7字符，则不能，而应使用符号表将长名映射到短名。

13.5

English: Explain the **purpose of open() and close() operations**.

Answer: open() informs **the system that the file will become active**; close() informs that **the file is no longer in active use by the user**.

中文: 解释open()和close()操作的目的。

答案: open()告知系统文件将变为活动；close()告知用户不再主动使用该文件。

13.6

English: In some systems, a **subdirectory can be read and written like ordinary files**. a.

Describe **protection problems**. b. Suggest a scheme.

Answer: a. If a user **modifies directory entries** (e.g., file location), they **could access other files, defeating protection**. b. **Do not allow direct write**; provide **system operations for directory modifications**.

中文: 有些系统中，子目录可以像普通文件一样读写。a. 描述保护问题。b. 提出解决方案。

答案: a. 若用户修改目录项（如文件位置），可能访问其他文件，破坏保护。b. 不允许直接写入，提供系统调用来修改目录。

13.7

English: Consider **a system supporting 5,000 users. Allow 4,990 to access one file**. a. How **specify in UNIX?** b. Suggest **another scheme more effective**.

Answer: a. (i) **ACL(access-control list) with all 4,990 names**; (ii) **Put them in a group and set group access (if group size allows)**. b. Use **universal access with ACL listing the 10 excluded users with no privileges**.

中文: 考虑支持5000用户的系统，允许其中4990个用户访问一个文件。a. 如何在UNIX中指定？b. 提出另一种更有效的方案。

答案: a. (i) 使用包含所有4990用户名的ACL；(ii) 将他们设为一个组并设置组访问权限（若组大小允许）。b. 使用通用访问，同时用ACL列出被排除的10个用户并赋予无权限。

13.8

English: Researchers suggested user control list (per user, which files they can access) instead of ACL per file. Discuss merits.

Answer: File-based ACL: easier to change permissions and less space. User-based: less overhead when opening a file.

中文: 研究者建议使用用户控制列表（每个用户列出可访问的文件）而非每个文件的ACL。讨论优点。

答案: 基于文件的ACL：更改权限更容易，占用空间少。基于用户的：打开文件时开销较小。

Chapter 14: File-System Implementation

14.1

English: Consider a file of 100 blocks. FCB/index block in memory. Calculate disk I/O operations for contiguous, linked, indexed allocation for adding/removing block at beginning/middle/end. (Table given)

- when moving for contiguous allocation, the disk cannot directly move. It need the memory reads the data and write again to corresponding position.
- indexed allocation: all pointers set stores in the memory.

Answer:

中文: 考虑一个100块的文件，FCB/索引块已在内存。计算连续、链接、索引分配下在文件开头/中间/末尾添加或删除一块所需的磁盘I/O次数。

答案: 添加开头：201,1,1；添加中间：101,52,1；添加末尾：1,3,1；删除开头：198,1,0；删除中间：98,52,0；删除末尾：0,100,0。

14.2

English: Why must the bit map for file allocation be kept on mass storage, rather than in main memory?

Answer: In case of system crash(memory failure), the free-space list would not be lost. As it would be if the bit map has been stored in main memory.

中文: 为什么文件分配的位图必须保存在大容量存储中，而不是主存？

答案:

14.3

English: Consider contiguous, linked, indexed allocation. What criteria decide which strategy is

best for a particular file?

Answer: Contiguous – if file is sequentially accessed and small. Linked – if large and sequentially accessed. Indexed – if large and randomly accessed.

中文： 考虑连续、链接、索引分配。决定哪种策略最适合特定文件的标准是什么？

答案： 连续——文件顺序访问且较小。链接——文件大且顺序访问。索引——文件大且随机访问。

14.4

English: One problem with contiguous allocation is preallocation. A solution: initial contiguous area plus linked overflow areas. Compare with standard contiguous and linked.

Answer: This method has more overhead than contiguous but less than linked. (time overhead. Continuous can use offset. Linked only access one by one)

中文： 连续分配的一个问题需要预分配。一种解决方案：初始连续区加链接溢出区。与标准连续和链接实现比较。

答案： 该方法开销大于标准连续分配，但小于标准链接分配。

14.5

English: How do caches help improve performance? Why not use more or larger caches?

Answer: Caches store data from slower devices in faster ones, improving speed. They are more expensive, so increasing size or number increases cost.

中文： 缓存如何帮助提高性能？为什么不使用更多或更大的缓存？

答案： 缓存将较慢设备的数据暂存于较快设备中，提高速度。缓存更昂贵，增加大小或数量会增加成本。

14.6

English: Why is it advantageous to dynamically allocate internal tables? What are the penalties?

Answer: Advantage: flexibility, no artificial limits. Penalties: more complex kernel code, more bug potential; dynamic tables use more resources, potentially taking resources from other parts.

中文： 为什么动态分配内部表有优势？有何代价？

答案： 优势：灵活性，无人为限制。代价：内核代码更复杂，易出错；动态表占用更多资源，可能影响其他部分。

Chapter 15: File-System Internals

15.1

English: Explain how the VFS layer allows an OS to support multiple file system types easily.

- VFS: virtual file system. A set of normal and standard file interfaces.

Answer: VFS provides indirection layer; generic system calls are translated at VFS to file-system-specific operations. The calling program has no file-system-specific code.

中文: 解释VFS层如何使操作系统轻松支持多种文件系统类型。

答案: VFS提供间接层; 通用系统调用在VFS层被转换为特定文件系统的操作。调用程序无需文件系统相关代码。

15.2

English: Why have more than one file system type on a given system?

Answer: Different file systems are optimized for specific uses (e.g., memory-based vs. secondary storage, or special-purpose like procfs).

中文: 为什么一个系统上要有多种文件系统类型?

答案: 不同文件系统针对特定用途优化 (如内存型 vs 辅助存储, 或特殊目的如procfs)。

15.3

English: On a Unix/Linux system with procfs, how to explore process name space? What aspects can be viewed? How to gather same info without procfs?

Answer: procfs mounted at /proc; each process has a directory by PID. Can view cwd, command line, priority, memory, locks, open files, etc. Without procfs, use separate system calls or open /dev/kmem and write programs to extract data.

中文: 在支持procfs的Unix/Linux系统上, 如何探索进程命名空间? 可查看哪些方面? 没有procfs如何获取相同信息?

答案: procfs挂载于/proc; 每个进程有PID目录。可查看当前工作目录、命令行、优先级、内存、锁、打开文件等。没有procfs时, 需使用单独的系统调用或打开/dev/kmem并编写程序提取数据。

15.4

English: Why do some systems integrate mounted file systems into the root naming structure, while others use a separate naming method?

Answer: Integration with root has proven more flexible and useful; separate mount point naming is less common. Choices often based on historical implementation details.

中文：为什么有些系统将挂载的文件系统集成到根命名结构中，而另一些使用单独的命名方法？

答案：与根集成已被证明更灵活有用；单独挂载点命名较少见。选择常基于历史实现细节。

15.5

English: Given remote file access like ftp, why were remote file systems like NFS created?

Answer: Users prefer seamless integration; file system interfaces are familiar and allow existing tools/scripts to work with remote files just as with local files.

中文：既然已有ftp等远程文件访问，为何还要创建NFS等远程文件系统？

答案：用户喜欢无缝集成；文件系统接口熟悉，允许现有工具/脚本像处理本地文件一样处理远程文件。

Chapter 17: Protection

17.1

English: What protection problems may arise if a shared stack is used for parameter passing?

Answer: The contents of the stack could be compromised by any other processes sharing the stack.

中文：如果使用共享栈传递参数，可能产生哪些保护问题？

答案：栈内容可能被共享该栈的其他进程破坏。

17.2

English: Consider a computing environment where a unique number is associated with each process and each object. Suppose process with number n can access object with number m only if $n > m$. What type of protection structure do we have?

Answer: A hierarchical structure.

中文：考虑每个进程和每个对象都有唯一编号的环境。假设编号为 n 的进程仅在 $n > m$ 时才能访问编号为 m 的对象。这是什么类型的保护结构？

答案：层次结构。

17.3

English: Consider a computing environment where a process is given the privilege of accessing an object only n times. Suggest a scheme for implementing this policy.

Answer: Add an integer counter with the capability.

中文： 考虑进程被授予仅访问某对象n次权限的环境。提出实现该策略的方案。

答案： 在能力（capability）中添加一个整数计数器。

17.4

English: If all access rights to an object are deleted, the object should be deleted. Suggest an efficient implementation.

Answer: Use reference counts. (0 to delete the object)

中文： 若对象的所有访问权被删除，对象也应被删除。提出一种高效实现。

答案： 使用引用计数。

17.5

English: Why is it difficult to protect a system in which users are allowed to do their own I/O?

Answer: I/O must be performed in kernel mode because it involves privileged operations. If users do their own I/O, they could bypass protection mechanisms.

中文： 为什么允许用户自行执行I/O的系统难以保护？

答案： I/O必须在内核模式下执行，因为它涉及特权操作。若用户自行I/O，可能绕过保护机制。

17.6

English: Capability lists are usually kept within the user's address space. How does the system ensure that the user cannot modify the list?

Answer: A capability list is considered a protected object and is accessed only indirectly by the user. The OS ensures the user cannot directly access it.

中文： 能力列表通常保存在用户地址空间中。系统如何确保用户不能修改该列表？

答案： 能力列表被视为受保护对象，用户只能间接访问。操作系统确保用户不能直接访问它。